CRM+Plugin（IDbConnection）扩展方法帮助文档

由 | 李钰萍 | 最近更新于 2023年5月15日在GMT+8 14:21

CRM Plugin（IDbConnection）扩展方法帮助文档

Plugin

前置

```
//获取配置文件路径取值：可以放在配置参数表里面
//参数
ConfigFactory config = new ConfigFactory(@"C:\CRMConfig\Vyung.Xrm.config");
```

同步

Execute

描述
Execute 是一个可以从 IDbConnection 类型的任意对象调用的扩展方法，它可以执行一个或多个命令并返回受影响的行数。此方法通常用于执行：

- 存储过程
- INSERT 语句
- UPDATE 语句
- DELETE 语句

参数
下表显示了 Execute 方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的命令文本。
param	命令参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

案例 - 执行存储过程
单次
执行一次存储过程。

```
string sql = "Invoice_Insert";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))😉
{
var affectedRows = connection.Execute(sql,
new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"},
commandType: CommandType.StoredProcedure);
}
```

多次
执行多次存储过程，为参数数组列表中的每个对象执行一次。

```
string sql = "Invoice_Insert";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))😉
{
connection.Open();

var affectedRows = connection.Execute(sql,
new[]
{
new {Kind = InvoiceKind.WebInvoice, Code = "Many_Insert_1"},
new {Kind = InvoiceKind.WebInvoice, Code = "Many_Insert_2"},
```

```
new {Kind = InvoiceKind.StoreInvoice, Code = "Many_Insert_3"}
},
commandType: CommandType.StoredProcedure
);
}
```

案例 - 执行 INSERT 语句

单次

执行一次 INSERT 语句。

```
string sql = "INSERT INTO Invoice (Code) Values (@Code);";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var affectedRows = connection.Execute(sql, new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"});
}
```

多次

执行 INSERT 语句，为参数数组列表中的每个对象执行一次。

```
string sql = "INSERT INTO Invoice (Code) Values (@Code);";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var affectedRows = connection.Execute(sql,
new[]
{
new {Kind = InvoiceKind.WebInvoice, Code = "Many_Insert_1"},
new {Kind = InvoiceKind.WebInvoice, Code = "Many_Insert_2"},
new {Kind = InvoiceKind.StoreInvoice, Code = "Many_Insert_3"}
}
);

}
```

案例 - 执行 UPDATE 语句

单次

执行一次 UPDATE 语句。

```
string sql = "INSERT INTO Invoice (Code) Values (@Code);";
using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var affectedRows = connection.Execute(sql, new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"});
```

}
多次
执行 UPDATE 语句，为参数数组列表中的每个对象执行一次。
string sql = "UPDATE Invoice SET Code = @Code WHERE InvoiceID = @InvoiceID";
using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString())😉 { var affectedRows = connection.Execute(sql, new[] { new {InvoiceID = 1, Code = "Many_Update_1"}, new {InvoiceID = 2, Code = "Many_Update_2"}, new {InvoiceID = 3, Code = "Many_Update_3"} }); }
案例 - 执行 DELETE 语句
单次
执行一次 DELETE 语句。
string sql = "DELETE FROM Invoice WHERE InvoiceID = @InvoiceID";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString())😉
{
var affectedRows = connection.Execute(sql, new {InvoiceID = 1});

}
多次
执行 DELETE 语句，为参数数组列表中的每个对象执行一次。
string sql = "DELETE FROM Invoice WHERE InvoiceID = @InvoiceID";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString())😉
{
var affectedRows = connection.Execute(sql,
new[]
{
new {InvoiceID = 1},
new {InvoiceID = 2},
new {InvoiceID = 3}
});
}

Query

描述
Query是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以执行查询并映射结果。
结果可以映射到：

- 匿名类型
- 强类型
- 多映射（一对一）
- 多映射（一对多）
- 多类型

参数
下表显示了Query方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
buffered	是否从缓冲读取查询结果（默认为true）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

案例 - 查询匿名类型
原生SQL查询可以使用Query方法执行，并将结果映射到动态类型列表。
string sql = "SELECT * FROM Invoice;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString())) { var invoices = connection.Query(sql).ToList(); //输出： 3 anonymous entity returned }
案例 - 查询强类型

原生SQL查询可以使用Query方法执行，并将结果映射到强类型列表。

```
string sql = "SELECT * FROM Invoice;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{

var invoices = connection.Query<Invoice>(sql).ToList();

//输出：3 invoice(s) returned
}
```

案例 - 查询多映射（一对一）

原生SQL查询可以使用Query方法执行，并将结果映射到具有一对一关系的强类型列表。

```
string sql = "SELECT * FROM Invoice AS A INNER JOIN InvoiceDetail AS B ON A.InvoiceID = B.InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{

var invoices = connection.Query<Invoice, InvoiceDetail, Invoice>(
sql,
(invoice, invoiceDetail) =>
{
invoice.InvoiceDetail = invoiceDetail;
return invoice;
},
splitOn: "InvoiceID")
.Distinct()
.ToList();

//输出：3 invoice(s) returned (Including InvoiceDetail information)
}
```

案例 - 查询多映射（一对多）

原生SQL查询可以使用Query方法执行，并将结果映射到具有一对多关系的强类型列表。

```
string sql = "SELECT * FROM Invoice AS A INNER JOIN InvoiceItem AS B ON A.InvoiceID = B.InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{

var invoiceDictionary = new Dictionary<int, Invoice>();
```

```
var invoices = connection.Query<Invoice, InvoiceItem, Invoice>(
sql,
(invoice, invoiceItem) =>
{
Invoice invoiceEntry;

if (!invoiceDictionary.TryGetValue(invoice.InvoiceID, out invoiceEntry))
{
invoiceEntry = invoice;
invoiceEntry.Items = new List<InvoiceItem>();
invoiceDictionary.Add(invoiceEntry.InvoiceID, invoiceEntry);
}

invoiceEntry.Items.Add(invoiceItem);
return invoiceEntry;
},
splitOn: "InvoiceID")
.Distinct()
.ToList();

//输出：3 invoice(s) returned (Including 6 InvoiceItem)
}
```

案例 - 查询多类型

原生SQL查询可以使用Query方法执行，并将结果映射到不同类型的列表。

```
string sql = "SELECT * FROM Invoice;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoices = new List<Invoice>();

using (var reader = connection.ExecuteReader(sql))
{
var storeInvoiceParser = reader.GetRowParser<StoreInvoice>();
var webInvoiceParser = reader.GetRowParser<WebInvoice>();

while (reader.Read())
{
Invoice invoice;
```

```
switch ((InvoiceKind) reader.GetInt32(reader.GetOrdinal("Kind")))
{
case InvoiceKind.StoreInvoice:
invoice = storeInvoiceParser(reader);
break;
case InvoiceKind.WebInvoice:
invoice = webInvoiceParser(reader);
break;
default:
throw new Exception(ExceptionMessage.GeneralException);
}

invoices.Add(invoice);
}
}

//输出：3 invoice(s) returned (StoreInvoice:1, WebInvoice:2)
}
```

QueryFirst

描述
QueryFirst是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以执行查询并映射第一个结果。
结果可以映射到：

- 匿名类型
- 强类型

参数
下表显示了QueryFirst方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

First, Single & Default
注意使用正确的方法。First和Single的方法是非常不同的。

结果	没有项	有一项	有多项
First	抛异常	当前项	第一项
Single	抛异常	当前项	抛异常
FirstOrDefault	默认值	当前项	第一项
SingleOrDefault	默认值	当前项	抛异常

案例 - 查询匿名类型
执行查询并将第一个结果映射到动态类型列表。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirst(sql, new {InvoiceID = 1});
```

```
}
```

案例 - 查询强类型

执行查询并将第一个结果映射到强类型列表。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirst<Invoice>(sql, new {InvoiceID = 1});
}
```

QueryFirstOrDefault

描述

QueryFirstOrDefault是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以执行查询并映射第一个结果，如果序列不包含任何元素则为默认值。

结果可以映射到：

- 匿名类型
- 强类型

参数

下表显示了QueryFirstOrDefault方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

First, Single & Default
注意使用正确的方法。First和Single的方法是非常不同的。

结果	没有项	有一项	有多项
First	抛异常	当前项	第一项
Single	抛异常	当前项	抛异常
FirstOrDefault	默认值	当前项	第一项
SingleOrDefault	默认值	当前项	抛异常

案例 - 查询匿名类型

执行查询并将第一个结果映射到动态类型列表，如果序列不包含任何元素则为默认值。

string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

```
using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirstOrDefault(sql, new {InvoiceID = 1});
}
```

案例 - 查询强类型

执行查询并将第一个结果映射到强类型列表，如果序列不包含任何元素则为默认值。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirstOrDefault<Invoice>(sql, new {InvoiceID = 1});
}
```

QuerySingle

描述
QuerySingle是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以执行查询并映射第一个结果，如果序列中没有元素则会引发异常。
结果可以映射到：

- 匿名类型
- 强类型

参数
下表显示了QuerySingle方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

First, Single & Default
注意使用正确的方法。First和Single的方法是非常不同的。

结果	没有项	有一项	有多项
First	抛异常	当前项	第一项
Single	抛异常	当前项	抛异常
FirstOrDefault	默认值	当前项	第一项
SingleOrDefault	默认值	当前项	抛异常

案例 - 查询匿名类型
执行查询并将第一个结果映射到动态类型列表，如果序列中没有元素则会引发异常。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QuerySingle(sql, new {InvoiceID = 1});
```

```
}
```

案例 - 查询强类型

执行查询并将第一个结果映射到强类型列表，如果序列中没有元素则会引发异常。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QuerySingle<Invoice>(sql, new {InvoiceID = 1});
}
```

QuerySingleOrDefault

描述

QuerySingleOrDefault是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以执行查询并映射第一个结果，如果序列为空则为默认值。如果序列中有多个元素，则此方法将引发异常。

结果可以映射到：

- 匿名类型
- 强类型

参数

下表显示了QuerySingleOrDefault方法的不同参数。

名称	描述

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

First, Single & Default
注意使用正确的方法。First和Single的方法是非常不同的。

结果	没有项	有一项	有多项
First	抛异常	当前项	第一项
Single	抛异常	当前项	抛异常
FirstOrDefault	默认值	当前项	第一项
SingleOrDefault	默认值	当前项	抛异常

案例 - 查询匿名类型
执行查询并将第一个结果映射到动态类型列表，如果序列为空则为默认值。如果序列中有多个元素，则此方法将引发异常。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
```

```
var invoice = connection.QuerySingleOrDefault(sql, new {InvoiceID = 1});
}
```

案例 - 查询强类型

执行查询并将第一个结果映射到强类型列表，如果序列为空则为默认值。如果序列中有多个元素，则此方法将引发异常。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QuerySingleOrDefault<Invoice>(sql, new {InvoiceID = 1});
}
```

QueryMultiple

描述

QueryMultiple是一个可以从IDbConnection类型的任意对象调用的扩展方法，它可以在相同的命令和映射结果中执行多个查询。

```
string sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID; SELECT * FROM InvoiceItem WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
using (var multi = connection.QueryMultiple(sql, new {InvoiceID = 1}))
{
var invoice = multi.Read<Invoice>().First();
var invoiceItems = multi.Read<InvoiceItem>().ToList();
}
}
```

参数
下表显示了QueryMultiple方法的不同参数。

| | |
| 名称 | 描述 |

sql	要执行的查询。
param	查询参数（默认为null）。
transaction	需要使用的事务（默认为null）。
commandTimeout	命令执行超时时间（默认为null）。
commandType	命令类型（默认为null）。

异步

使用了Async（异步）方法扩展了IDbConnection接口：

- ExecuteAsync
- QueryAsync
- QueryFirstAsync
- QueryFirstOrDefaultAsync
- QuerySingleAsync
- QuerySingleOrDefaultAsync
- QueryMultipleAsync

我们只在本教程中添加了非异步版本，以便于阅读。

ExecuteAsync

```
var sql = "Invoice_Insert";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var affectedRows = connection.ExecuteAsync(sql,
new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"},
commandType: CommandType.StoredProcedure)
```

```
.Result;
}
```

QueryAsync

```
var sql = "Invoice_Insert";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoices = connection.QueryAsync<Invoice>(sql).Result.ToList();
}
```

QueryFirstAsync

```
var sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirstAsync<Invoice>(sql, new {InvoiceID = 1}).Result;
}
```

QueryFirstOrDefaultAsync

```
var sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QueryFirstOrDefaultAsync<Invoice>(sql, new {InvoiceID = 1}).Result;
}
```

QuerySingleAsync

```
var sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QuerySingleAsync<Invoice>(sql, new {InvoiceID = 1}).Result;
}
```

QuerySingleOrDefaultAsync

```
var sql = "SELECT * FROM Invoice WHERE InvoiceID = @InvoiceID;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoice = connection.QuerySingleOrDefaultAsync<Invoice>(sql, new {InvoiceID = 1}).Result;
}
```

QueryMultipleAsync

```
var sql = "SELECT * FROM Invoice; SELECT * FROM InvoiceItem;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
using (var multi = connection.QueryMultipleAsync(sql, new { InvoiceID = 1 }).Result)
{
var invoice = multi.Read<Invoice>().First();
var invoiceItems = multi.Read<InvoiceItem>().ToList();
}
}
```

缓冲

描述

- 默认值：True

缓冲查询一次返回整个读取器，这在大多数情况下是理想的。
非缓冲查询与流式传输等效，您只需按需加载对象，这对于一个非常大的查询来减少内存使用情况可能很有用。

```
string sqlInvoices = "SELECT * FROM Invoice;";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
var invoices = connection.Query<Invoice>(sqlInvoices, buffered: false).ToList();
}
```

事务

描述

支持事务和范围事务。必须要手动Open：connection.Open()

事务

从连接开始一个新事务，并将其传递给事务可选参数。

```
var sql = "Invoice_Insert";

using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString()))
{
connection.Open();

using (var transaction = connection.BeginTransaction())
{
var affectedRows = connection.Execute(sql,
new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"},
commandType: CommandType.StoredProcedure,
transaction: transaction);

transaction.Commit();
}
}
```

范围事务

在开始连接之前开始一个新的范围事务。

```
// using System.Transactions;
using (var transaction = new TransactionScope()) { var sql = "Invoice_Insert"; using (IDbConnection connection = new SqlConnection(config.GetDBConnectionString())) { connection.Open(); var affectedRows = connection.Execute(sql, new {Kind = InvoiceKind.WebInvoice, Code = "Single_Insert_1"}, commandType: CommandType.StoredProcedure); } transaction.Complete(); }
```